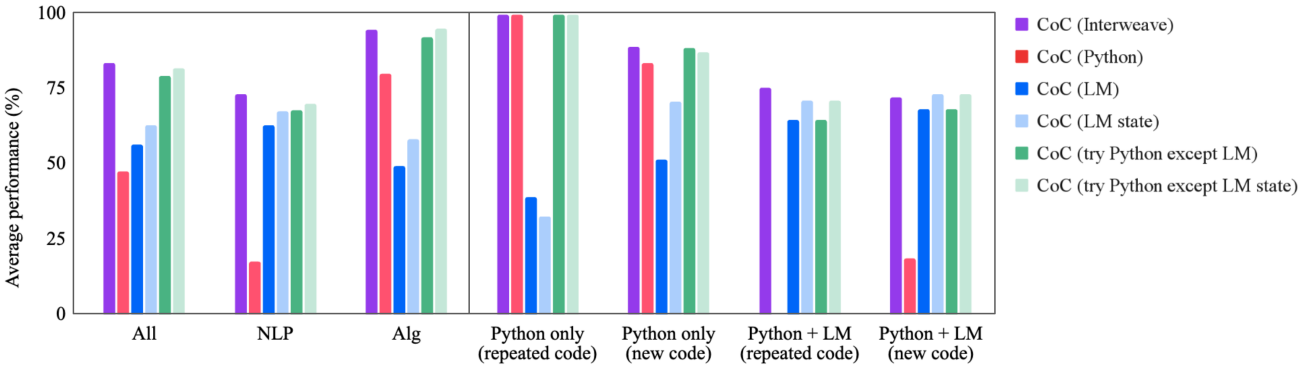

Figure 5. Chain of Code ablations on average performance grouped by task type.

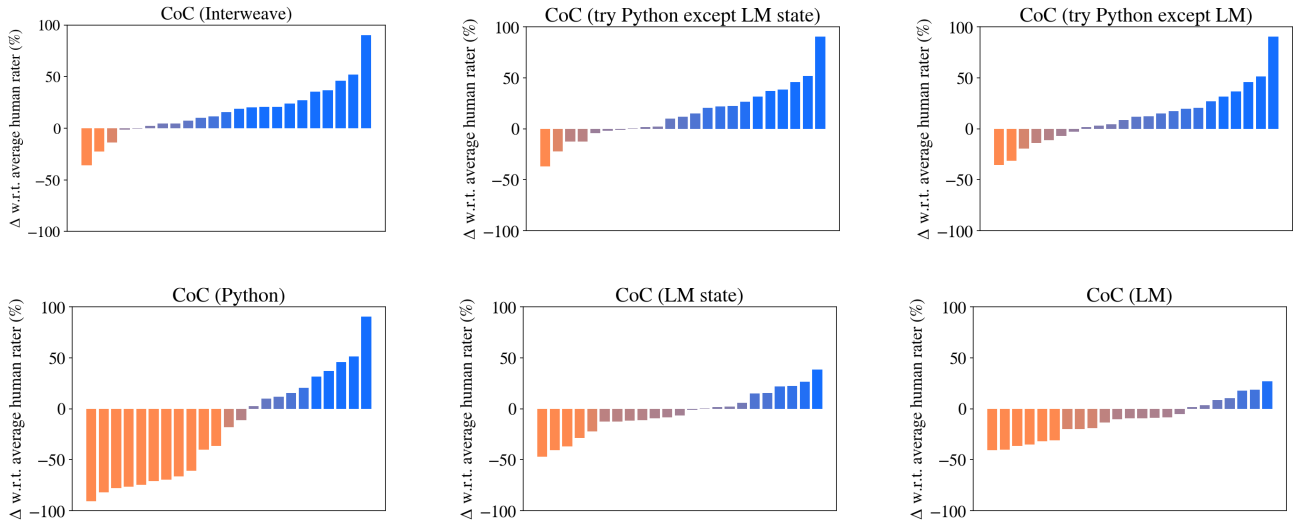

Figure 6. Results across all BIG-Bench Hard tasks compared to human baseline (Srivastava et al., 2022). The tasks (x-axis) in each plot are sorted individually by performance. See Table A1 and Figure 5 for a breakdown by task type.

Table 2. Ablation overall performance (%) with both few-shot prompting with a single task and cross-task. The delta compared to the full model (Interweave) is shown in parenthesis.

	Chain of Code					
Prompt	Interweave	try Python except LM state	try Python except LM	Python	LM state	LM
Single task	84	82 (-2)	80 (-4)	48 (-36)	63 (-21)	57 (-27)
Cross task	61	57 (-4)	60 (-1)	35 (-26)	49 (-12)	50 (-11)

former is more amenable to few-shot prompting with examples, which is the main evaluation paradigm for BIG-Bench Hard. However, we still made our best attempt to evaluate our method with the instruction tuned models using two different setups. The first is zero-shot prompting, where we directly prompt the models via the system message to output direct answers, chain of thoughts, or pseudocode/code (which we optionally execute with the python interpreter and feed back the results). The second is few-shot prompting, where we coerce the models to behave like completion models via the system message, and feed the few-shot examples as usual. In both cases, we demonstrated that CoC brings noticeable benefits with little modification needed. See Sec. A.4 for more details.

Question 7: Robustness of Chain of Code We showed that CoC is generally robust against prompt variation by evaluating with different prompts independently written by three annotators on the same set of problems. Specifically, we select four representative tasks from BIG-Bench Hard that require generation of new code (as opposed to repeated code). While the performance of individual tasks has some variance, the average performance across the four tasks only vary within a few percentage points. See Sec. A.5 for more details.

Question 8: Beyond Language Reasoning We showed that

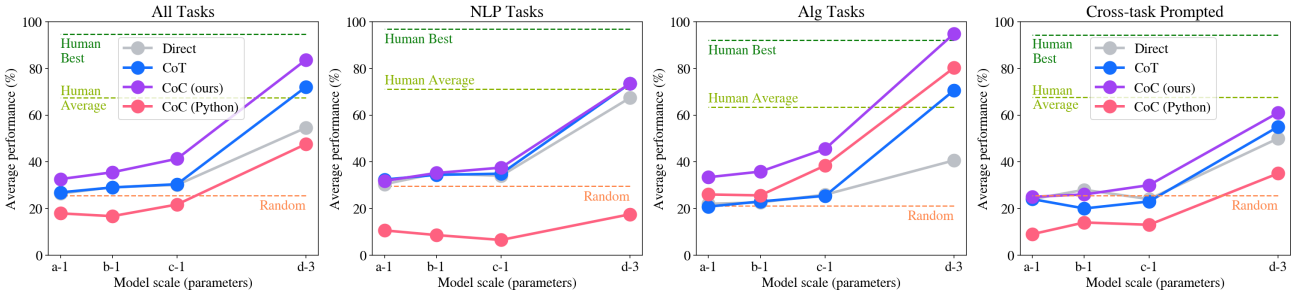

Figure 7. Average performance with model scaling, from `text-ada-001` (smallest) to `text-davinci-003` (largest).

CoC is well-suited for tasks that require both semantic and algorithmic reasoning beyond language reasoning, such as robotics. The unique advantage of CoC in robotics is that it interact seamlessly with the robot perception and control APIs via python code such as running object detectors or invoking parameterized robot skills, while performing semantic subtasks in an "inline" fashion (e.g. classifying what trash is compostable before picking them). When equipped with the necessary robot APIs, and a single example in the prompt to teach LMs the format, CoC can solve seven different robot manipulation tasks in the real world, showcasing generalization to new objects, languages and task domains. See Sec. A.6 for more details.

4. Related Work

Language Model Reasoning The abilities and applications of language models have seen significant progress, due to their overall performance (Chowdhery et al., 2022; Touvron et al., 2023; Radford et al., 2019; Gemini Team, 2023) and emergent capabilities (Wei et al., 2022a), such as few-shot prompting (Brown et al., 2020) and abstract reasoning (Wei et al., 2022b). Perhaps most related to this work, a number of works have leveraged prompting to improve reasoning (Dohan et al., 2022): Chain of Thought (Wei et al., 2022b) proposes to break a task down into intermediate reasoning steps, least-to-most (Zhou et al., 2022a) proposes a series of increasingly simpler problems, and ScratchPad (Nye et al., 2021) proposes to maintain a trace of intermediate results for interpreting code (this first demonstrated the code simulation ability of LMs required for our LMulator). Along these lines "let's think step by step" (Kojima et al., 2022) uses a few key words to elicit such break downs (words that were later refined to "Take a deep breath and work on this problem step-by-step" in (Yang et al., 2023)). Beyond these, other approaches structure such step-by-step solutions into graphical structures (Yao et al., 2023; Besta et al., 2023), plans (Wang et al., 2023b; Ning et al., 2023), or mixture of expert-based sampling (Wang et al., 2022; Zhou et al., 2022b). CoC builds upon the intuition of these works, with the observation that *code* is a formal, structured approach to breaking a problem down into sub-steps with many advantages beyond natural language alone.

Language Model Tool Use Many recent works have proposed techniques for language models to use tools to respond to queries (Mialon et al., 2023). These tools have often been provided to the language model through prompting (Cobbe et al., 2021; Khot et al., 2022; Chowdhery et al., 2022; Drori et al., 2022; Yao et al., 2022), enabling tools like calculators for math problems, code interpreters, databases, or more. These tools too can provide feedback on novel modalities (Surís et al., 2023; Zeng et al., 2022). To expand the range of tools available, others have used external tool databases or finetuned language models (Schick et al., 2023; Qin et al., 2023; Parisi et al., 2022; Paranjape et al., 2023). As tool interfaces vary, feedback from the tool too can improve performance (Gou et al., 2023; Zhou et al., 2023). In this work we leverage the expressibility and generality of full code as well as its structure, by treating it both as a tool and as a framework.

Language Model Program Synthesis The ability of language models to code is well known and they have been applied as programming assistants (Chen et al., 2021) and shown to be capable programmers on their own (Austin et al., 2021; Li et al., 2022; Nijkamp et al., 2022). This ability has been applied to a variety of tasks outside of language alone, leveraging their ability to reason through code in new settings, such as robotics (Liang et al., 2023; Singh et al., 2023), embodied agents (Wang et al., 2023a), or vision (Surís et al., 2023). Others have specifically done so for reasoning, such as Program of Thoughts (Chen et al., 2022) and Program-aided Language Models (Gao et al., 2023), which generate code to solve numerical reasoning problems. Herein, we focus on the interplay between writing code, running code, and language models simulating code, thus enabling new regimes of language model code applications, such as semantic reasoning.

5. Conclusions, Limitations, and Future Work

We have proposed Chain of Code, an approach towards reasoning with language models through writing code, and executing code either with an interpreter or with a language model that simulates the execution (termed herein an LMulator) if the code is not executable. As such, CoC can leverage both the expressive structure of code and the powerful tools available to it. Beyond this, by simulating the execution of non-executable code, CoC can apply to problems nominally

outside the scope of code (e.g., semantic reasoning problems). We have demonstrated that this approach outperforms baselines, and for some tasks even the best human raters, in a range of challenging language and numeric reasoning problems.

This work is not without its limitations. First, generating and executing in two steps as well as interweaving code and language execution requires additional context length and computation time. Second, though we have not seen any loss of performance for semantic tasks in aggregate, there are few tasks in which code doesn't help, e.g., the task Ruin Names, which asks whether an edit for a name is humorous. Finally, our implementation to interweave LM and code is quite simple, tracking the program state in strings and parsing the strings into Python's built-in data types (e.g., dict, tuple). As our method stands now, the LM cannot modify custom Python objects while simulating code execution. In theory, however, it is doable as long as each of these Python objects have a serialization and deserialization method, e.g., using techniques like Protocol Buffers.

There are many avenues for future work with CoC. First, we believe that a unified code and language interpreter well combines the commonsense of language models with the analytical abilities, structure, and interpretability of code. Such a technology can thus enable applications of code and code-like reasoning to novel problem regimes, beyond simple reasoning. Second, we are interested in investigating the degree to which finetuning a language model to be an LMulator can benefit semantic code reasoning. Third, we see evidence that reasoning through many pathways yields improvements, which is a promising step forward. Finally, we believe this integration with code enables access to external modalities, such as vision or databases, and represents a interesting path for new applications (e.g., robotics, augmented reality).

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, most of which are related to the usage of large language models (LLMs). One aspect of Chain of Code that warrants further discussion is that CoC executes the output of LLMs using the Python interpreter as if they are always benign code. If deployed in the wild, however, Chain of Code will need to install additional safeguards against potentially harmful code from LLMs that might be maliciously prompted, before running the code.

References

Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

Besta, M., Blach, N., Kubicek, A., Gerstenberger, R., Gianinazzi, L., Gajda, J., Lehmann, T., Podstawski, M., Niewiadomski, H., Nyczyk, P., et al. Graph of thoughts: Solving elaborate problems with large language models. *arXiv preprint arXiv:2308.09687*, 2023.

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. d. O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Chen, W., Ma, X., Wang, X., and Cohen, W. W. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *arXiv preprint arXiv:2211.12588*, 2022.

Chowdhery, A., Narang, S., Devlin, J., Bosma, M., Mishra, G., Roberts, A., Barham, P., Chung, H. W., Sutton, C., Gehrmann, S., et al. Palm: Scaling language modeling with pathways. *arXiv preprint arXiv:2204.02311*, 2022.

Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Dohan, D., Xu, W., Lewkowycz, A., Austin, J., Bieber, D., Lopes, R. G., Wu, Y., Michalewski, H., Saurous, R. A., Sohl-Dickstein, J., et al. Language model cascades. *arXiv preprint arXiv:2207.10342*, 2022.